LAB-9: Design Analysis Report

Name: Gyan Prakash Tiwari
Roll No.: 202401100200146
Branch: CSE

Project: AI Chatbot for University Helpdesk (RAG-based) |
github.com/Xxierrr/AI-Chatbot-for-University-Helpdesk

1. Aim

To analyse the design of the University Helpdesk Chatbot with respect to its architecture, design patterns, design principles, coupling and cohesion, quality attributes and risks, and to recommend improvements.

2. System Overview

The system is a web application that answers student questions on admissions, courses, facilities and campus life using **Retrieval-Augmented Generation (RAG)**. Administrators upload official documents which are chunked and embedded into ChromaDB; at query time the most relevant chunks are retrieved and passed to an LLM, which produces a grounded answer. Students and admins use a React front end; a FastAPI back end handles authentication, chat, analytics and document management.

3. Architectural Style

- **Client-server** with a single-page application talking to a REST API.
- **Layered back end:** routes (presentation) → services (business logic: rag_service, auth_service) → models/data access (SQLAlchemy) → storage (SQLite, ChromaDB).
- **Pipeline architecture** for RAG: ingest, chunk, embed, store; and query, retrieve, augment, generate.

4. Design Patterns Identified

Pattern	Where it appears	Benefit
Layered / Service Layer	api/routes call services/*.py instead of containing logic.	Separation of concerns, testability.
Facade	rag_service hides embedding, retrieval, prompt building and LLM call behind one method.	Simple interface for routes.
Dependency Injection	FastAPI `Depends()` for DB session and current user.	Loose coupling, easy mocking in tests.
Data Mapper / Repository	SQLAlchemy models separate the object model from SQL tables.	Database independence.
Strategy (configurable)	LLM provider chosen through environment variables (OpenAI or compatible).	Swap providers without code changes.
Role-Based Access Control	Student vs Admin checks on /api/admin/* routes.	Security and least privilege.

5. Design Principles (SOLID) Evaluation

Principle	Assessment
Single Responsibility	Good: auth, RAG and ingestion live in separate modules. Watch that route files do not grow business logic.
Open/Closed	Moderate: new document formats or LLM providers should be added through new loader/provider classes rather than editing if/else blocks.
Liskov Substitution	Applicable if LLM or vector-store wrappers share a common interface; any replacement must honour it.
Interface Segregation	Good: clients use only the endpoints they need (separate /api/admin/* group).
Dependency Inversion	Partial: services should depend on abstractions (retriever, LLM client) rather than concrete OpenAI/Chroma classes.

6. Coupling and Cohesion

- **Cohesion:** High functional cohesion in rag_service (everything relates to answering a question) and auth_service (identity only).
- **Coupling:** Mostly data coupling through well-defined REST/JSON and Pydantic schemas. The main concentration of coupling is the RAG service, which depends on the embedding model, ChromaDB and the LLM API.
- **Front-end/back-end coupling** is limited to the API contract, which keeps the UI replaceable.

7. Quality Attribute Analysis

Attribute	Strengths	Weaknesses / Risks
Security	bcrypt password hashing, JWT, RBAC, restricted CORS.	Default admin credentials; no rate limiting/lockout mentioned; prompt injection through uploaded documents.
Scalability	Stateless JWT allows multiple API instances.	SQLite has limited concurrent writes; local ChromaDB is single-node.
Performance	Small local embedding model; vector search is fast.	LLM latency dominates; no response caching.
Maintainability	Clear folder structure, modular services, config through .env.	Hard-coded paths in documentation; tests only listed, coverage unknown.
Reliability	Fallback path for unanswered questions.	Dependence on an external LLM; no retry or timeout policy described.
Usability	Typing indicator, suggestions, Markdown, history, feedback.	English only (multilingual is future work).

8. Recommendations

- Force a password change for the default admin on first login or generate credentials at install time.
- Introduce abstract interfaces (*LLMClient*, *VectorStore*, *DocumentLoader*) to fully satisfy Open/Closed and Dependency Inversion.
- Move from SQLite to PostgreSQL and use a server-mode vector database for production.
- Add rate limiting, request timeouts, retries and structured logging around the LLM call.
- Sanitise uploaded text and instruct the LLM to ignore instructions found inside documents.
- Raise automated test coverage (pytest + frontend tests) and run it in CI.

9. Conclusion

The design is modular, layered and well suited to a RAG helpdesk. Its main strengths are separation of concerns and replaceable infrastructure; the main improvements needed are stronger production hardening (credentials, rate limits, database) and deeper use of abstractions.

Note: this analysis is based on the project's documented architecture, folder structure and technology stack.